

C. IMPLEMENTASI ISR OTOMATIS

Bagian 1 – Tambahkan revalidate

Buka halaman static.tsx pada folder src/pages/produk

```
pemograman_framework - static.tsx

22  return {
23    props: {
24      products: response.data
25    },
26    revalidate: 10, // Revalidate data setiap 10 detik
27  }
```

Artinya:

- Setiap 10 detik halaman akan dicek ulang
- Jika ada perubahan data → cache diperbarui

Bagian 2 – Pengujian ISR

Jalankan: (lakukan hal sama seperti JS sebelumnya untuk ngebuild SSG)

```
• /produk/server
• /produk/static (368 ms) 10s 1y
• /profil
```

Tambahkan data baru di database pada firebase

Add a document

Parent path
/products

Document ID
WRUEmcj4Y9Ret4vmzh7

Field Type

name string

String
Slides Adilette Ayoon

Field Type Value

price number 900000

Field Type

image string

String
https://www.adidas.co.id/fd/slides-adilette-ayoon/GX7064.htm

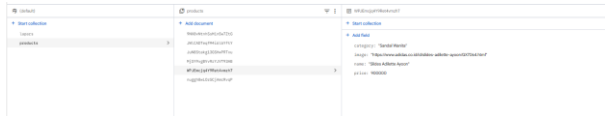
Field Type

category string

String
Sandal Wanita

⊕ Add field

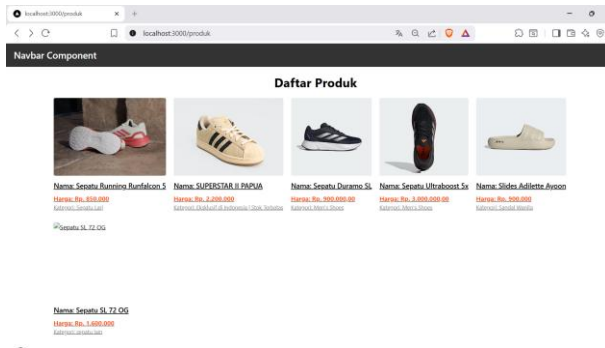
Cancel Save



Refresh halaman sebelum 10 detik → Data lama.

o Sebelum 10 detik data yang akan ditampilkan masih data lama

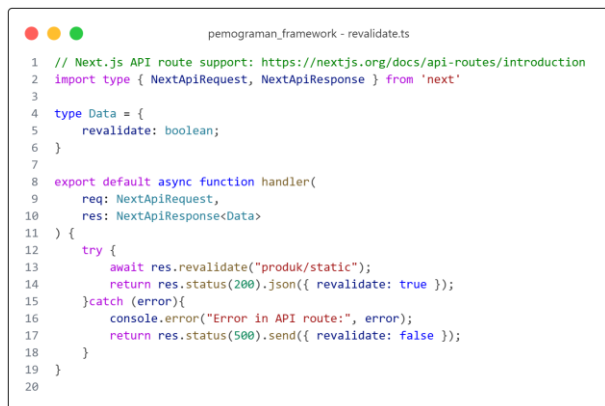
Refresh setelah 10 detik → Data baru muncul



D. ON-DEMAND REVALIDATION

Bagian 1 – Buat API Revalidate

Buat file revalidate.ts pada folder pages/api/ dan modifikasi



Note : jika kita menggunakan code ini ada sedikit masalah yaitu Ketika salah satu data dihapus maka data yang terhapus tidak secara otomatis terupdate pada layar meskipun sudah di refresh selama 10 detik. Untuk bisa terupdate data yang ditampilkan diperlukan trigger dengan mengakses apinya <http://localhost:3000/api/produk>

Bagian 2 – Tambahkan Parameter Data

Untuk mengatasi hal tersebut (pada bagian 1) maka suatu kondisi pada file revalidate.ts

o Modifikasi file revalidate.ts

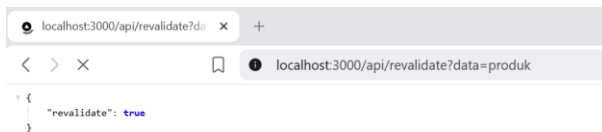
```

1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse } from "next";
3
4 type Data = {
5   revalidate: boolean;
6   message?: string;
7 };
8
9 export default async function handler(
10   req: NextApiRequest,
11   res: NextApiResponse<Data>,
12 ) {
13   if (req.query.data === "produk") {
14     try {
15       await res.revalidate("/produk/static");
16       return res.status(200).json({ revalidate: true });
17     } catch (error) {
18       console.error("Error in API route:", error);
19       return res.status(500).send({ revalidate: false });
20     }
21   }
22
23   return res.json({
24     revalidate: false,
25     message: "Invalid query parameter. Expected 'data=produk'.",
26   });
27 }
28

```

Uji coba menambahkan parameter dan value pada url

<http://localhost:3000/api/revalidate?data=produk> maka akan muncul true dan sesuai dengan kondisi (req.query.data === "produk")

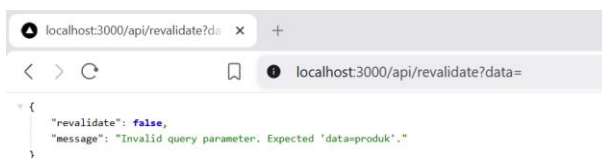


```

{
  "revalidate": true
}

```

Uji coba dengan url <http://localhost:3000/api/revalidate?data=>



```

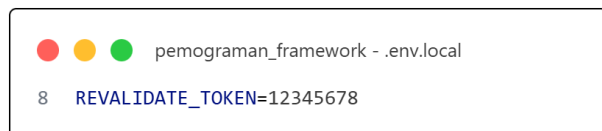
{
  "revalidate": false,
  "message": "Invalid query parameter. Expected 'data=produk'."
}

```

Bagian 3 – Tambahkan Token Security

Perlu ditambahkan paramater agar user tidak merubah data melalui url

Buka file .env dan modifikasi



```

8 REVALIDATE_TOKEN=12345678

```

Modifikasi file revalidate.ts tambahkan kondisi pada line 13 – 17

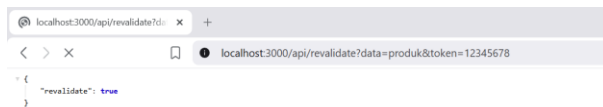
```
pemograman_framework - revalidate.ts
13 if(req.query.token !== process.env.REVALIDATE_TOKEN) {
14   return res.status(401).json({
15     revalidate: false,
16     message: "Insert correct token",
17   });
18 }
```

E. PENGUJIAN MANUAL REVALIDATION

Akses:

<http://localhost:3000/api/revalidate?data=products&token=12345678>

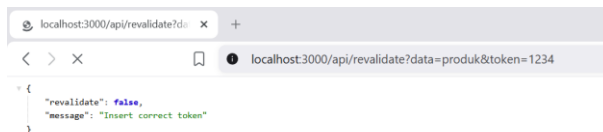
Jika benar:



The screenshot shows a web browser with the address bar at `localhost:3000/api/revalidate?data=produk&token=12345678`. The response body is a JSON object: `{ "revalidate": true }`.

`{ revalidate: true }`

Jika token salah:



The screenshot shows a web browser with the address bar at `localhost:3000/api/revalidate?data=produk&token=1234`. The response body is a JSON object: `{ "revalidate": false, "message": "Insert correct token" }`.

F. PERBANDINGAN SSG VS ISR

Aspek	SSG	ISR
Update Data	Harus build ulang	Otomatis / Trigger
Cache	Static permanen	Static + Refresh
Cocok untuk	Konten tetap	Konten semi-dinamis

G. TUGAS PRAKTIKUM

Tugas Individu

1. Tambahkan lagi produk pada firebase

Add a document

Parent path
/lapars

Document ID ⓘ
g0ZgBbWmFdlcF0LqXJ77

Field

name

=

Type

string

⊖

String

Kentak

Field

price

=

Type

number

Value

15000

⊖

Field

image

=

Type

string

⊖

String

https://encrypted-tbn0.gstatic.com/images?q=tbn:AND9GcTm3Vtr4G-dvBiBZSDREaq_lpmU_VffhZfOTQ&s

Field

category

=

Type

string

⊖

String

lawuh

⊕ Add field

Cancel Save

2. Implementasikan ISR dengan revalidate: 10.

Pages/lapar/static.tsx

```
22 return {  
23   props: {  
24     lapars: response.data  
25   },  
26   revalidate: 10,  
27 }
```

Build

```
• /lapar/static (621 ms) 10s 1y
```

3. Tambahkan endpoint On-Demand Revalidation.

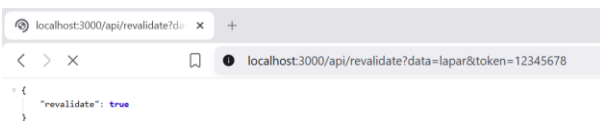
```
pemograman_framework - revalidate.ts
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse, NextApiRequest } from "next";
3
4 type Data = {
5   revalidate: boolean;
6   message?: string;
7 };
8
9 export default async function handler(
10   req: NextApiRequest,
11   res: NextApiResponse<Data>,
12 ) {
13
14   const data = req.query.data;
15
16   if (data === "produk" || data === "lapar") {
17     const path = data === "produk" ? "/produk/static" : "/lapar/static";
18
19     try {
20       await res.revalidate(path);
21       return res.status(200).json({ revalidate: true });
22     } catch (error) {
23       console.error("Error in API route:", error);
24       return res.status(500).send({ revalidate: false });
25     }
26   }
27
28   return res.json({
29     revalidate: false,
30     message: "Invalid query parameter. Expected 'data=produk' or 'data=lapar'.",
31   });
32 }
33
```

4. Tambahkan validasi token.

```
pemograman_framework - revalidate.ts
13 if(req.query.token !== process.env.REVALIDATE_TOKEN) {
14   return res.status(401).json({
15     revalidate: false,
16     message: "Insert correct token",
17   });
18 }
```

5. Uji dengan:

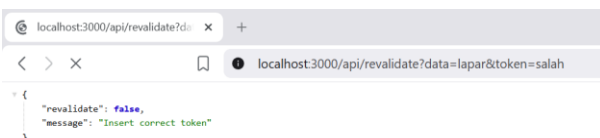
o Token benar



```
{
  "revalidate": true
}
```

"revalidate": **true**

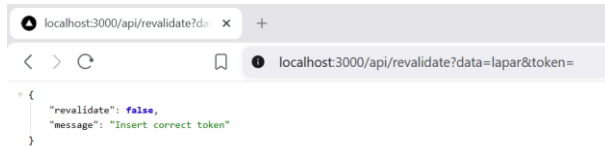
o Token salah



```
{
  "revalidate": false,
  "message": "Insert correct token"
}
```

"revalidate": **false**, "message": "Insert correct token"

o Tanpa token



```
{
  "revalidate": false,
  "message": "Insert correct token"
}
```

Sama dengan kondisi token salah

H. PERTANYAAN ANALISIS

1. Mengapa ISR lebih fleksibel dibanding SSG?

Karena bisa update konten pada halaman statis secara berkala tanpa harus melakukan rebuild seluruh aplikasi dari awal.

2. Apa perbedaan revalidate waktu dan on-demand?

Pemicunya, revalidate waktu terjadi otomatis setelah interval tertentu, sedangkan on-demand dipicu secara manual melalui API endpoint saat data berubah.

3. Mengapa endpoint revalidation harus diamankan?

Agar tidak ada yang memicu proses *rebuild* terus-menerus yang dapat membebani server.

4. Apa risiko jika token tidak digunakan?

Serangan DoS (Denial of Service) di mana penyerang bisa sengaja menghabiskan resource server dengan melakukan revalidasi tanpa henti.

5. Kapan ISR lebih cocok dibanding SSR?

Saat ada halaman dengan data yang jarang berubah namun memiliki traffic tinggi, karena ISR menghasilkan file statis yang lebih cepat dan ringan daripada merender ulang di setiap request.

LAPORAN JOBSHEET 13: MIDDLEWARE ROUTE PROTECTION MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

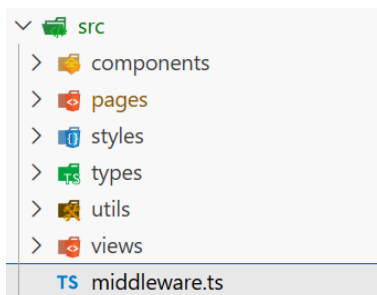
C. LANGKAH PRAKTIKUM

Bagian 1 – Membuat Middleware

Modifikasi file index.tsx pada folder src/pages/produk

```
pemograman_framework - index.tsx
1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 import TampilanProduk from "../../views/product";
4 import useSWR from "swr";
5 import fetcher from "../../utils/swr/fetcher";
6 // const fetcher = (url: string) => fetch(url).then((res) => res.json());
7 const kategori = () => {
8   // const [isLogin, setIsLogin] = useState(false);
9   const {push} = useRouter();
10  const [products, setProducts] = useState([]);
11  // Menggunakan SWR
12  const {data, error, isLoading} = useSWR("/api/produk", fetcher);
13
14  // cek apakah data, error, atau isLoading sudah benar
15  // console.log("Data:", data);
16  // console.log("Error:", error);
17  // console.log("Is Loading:", isLoading);
18
19  return (
20    <div className="container mx-auto p-4">
21      <TampilanProduk products={isLoading ? [] : data.data} />
22    </div>
23  );
24 };
25 export default kategori;
```

Buat file: src/middleware.ts Seajar dengan folder pages.



Bagian 2 – Struktur Dasar Middleware

```
pemograman_framework - middleware.ts
1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3
4 export function middleware(request: NextRequest) {
5   return NextResponse.next();
6 }
```

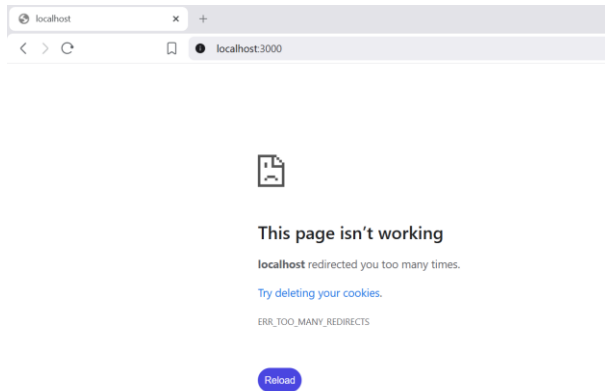
o Jika menggunakan NextResponse.next() → tidak ada redirect.

o Jadi masih bisa mengakses ke <http://localhost:3000/produk>

Bagian 3 – Redirect Sederhana

```
pemograman_framework - middleware.ts
1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3
4 export function middleware(request: NextRequest) {
5   return NextResponse.redirect(new URL("/", request.url));
6
7   // return NextResponse.next();
8 }
```

Semua halaman akan redirect ke home dan error dikarenakan terus menerus loading



Bagian 4 – Batasi Route Tertentu

```
pemograman_framework - middleware.ts
1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3
4 export function middleware(request: NextRequest) {
5   return NextResponse.redirect(new URL("/", request.url));
6   // return NextResponse.next();
7 }
8
9 export const config = {
10   matcher: ["/produk", "/about"],
11 }
```

Artinya:

- Middleware hanya berlaku untuk /products dan /about
- Halaman lain tetap normal
- Ketika user mengakses halaman produk dan about maka akan langsung redirect ke halaman home

Bagian 5 – Simulasi Sistem Login

o Modifikasi file middleware.ts

```

1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3
4 export function middleware(request: NextRequest) {
5   const isLogin = false;
6   if(isLogin){
7     return NextResponse.next();
8   }else{
9     return NextResponse.redirect(new URL("/auth/login", request.url));
10  }
11
12  // return NextResponse.redirect(new URL("/", request.url));
13  // return NextResponse.next();
14 }
15
16 export const config = {
17   matcher: ["/produk", "/about"],
18 }

```

Jika user langsung mengakses ke alamat <http://localhost:3000/produk> tidak akan bisa
user akan diarahkan ke halaman login

D. PENGUJIAN

Uji 1 – isLogin = false

```

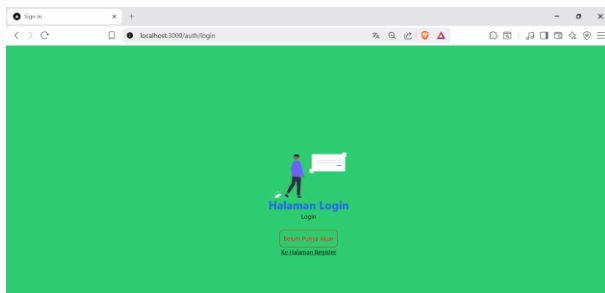
5 const isLogin = false;

```

Akses:

</products>

Hasil: Redirect ke /login



Benar

Uji 2 – isLogin = true

```

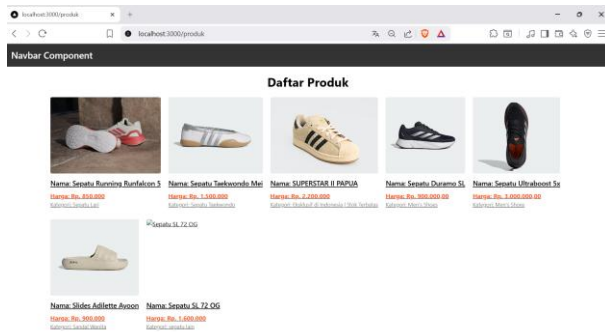
5 const isLogin = true;

```

Ubah:

`const isLogin = true`

Hasil: Bisa mengakses /products



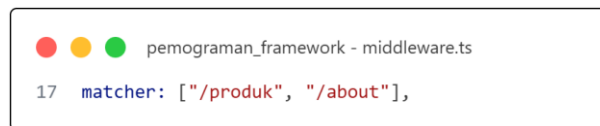
Benar

Uji 3 – Tambahkan Multiple Route

```
export const config = {
```

```
  matcher: ['/products', '/about']
```

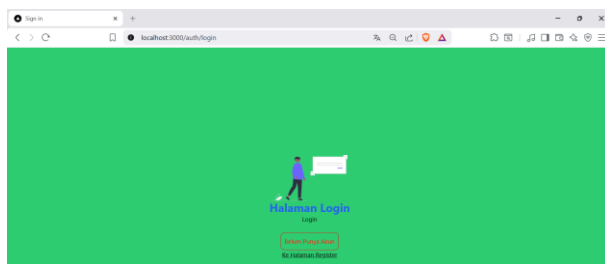
```
}
```



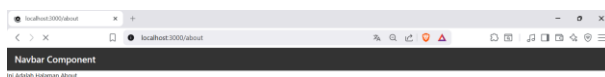
Sekarang:

- /products dan /about butuh login

Set isLoggedIn = false



Setelah isLoggedIn diset true



benar

- Halaman lain bebas



Benar. Contoh /lapar.

E. PERBANDINGAN MIDDLEWARE VS USEEFFECT

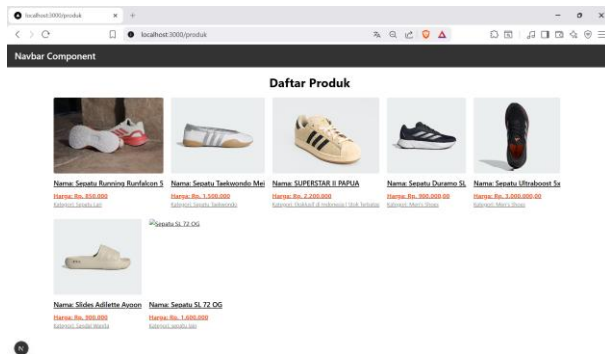
Aspek	useEffect	Middleware
Redirect timing	Setelah render	Sebelum render
Glitch	Ada	Tidak
Security	Lemah	Lebih aman
Skalabilitas	Harus tiap halaman	Sekali di middleware

F. TUGAS PRAKTIKUM

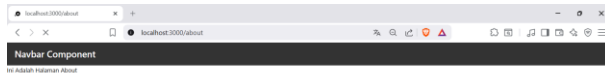
Tugas Individu

1. Buat halaman:

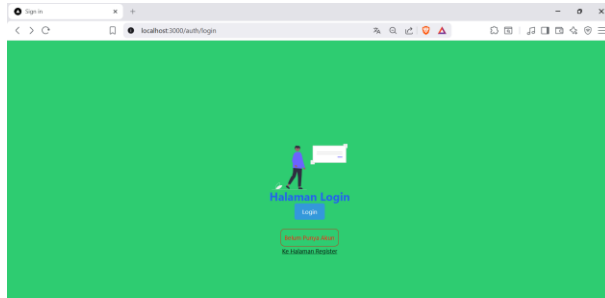
o /products



o /about



o /login



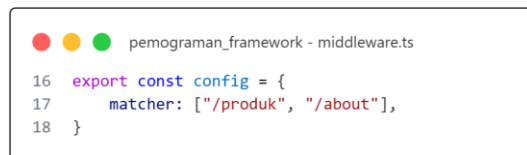
2. Implementasikan Middleware:

o Redirect ke /login jika belum login.

o Izinkan akses jika login true.



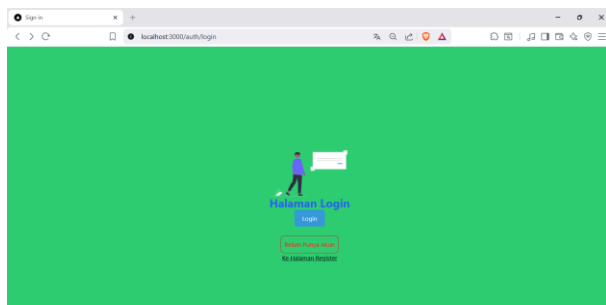
3. Tambahkan proteksi hanya untuk route tertentu.



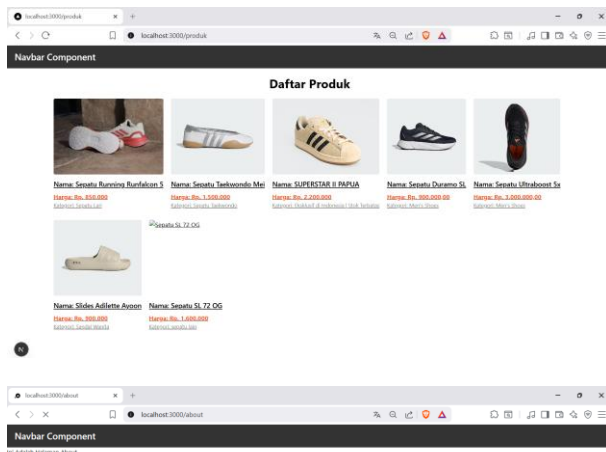
4. Dokumentasikan:

o Screenshot sebelum dan sesudah redirect.

Sebelum redirect



Sesudah redirect bisa akses (note: hanya jika isLogin diset true pada middleware)



o Perbandingan dengan useEffect.

Middleware

```

pemograman_framework - middleware.ts

1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3
4 export function middleware(request: NextRequest) {
5   const isLogin = true;
6   if(isLogin){
7     return NextResponse.next();
8   }else{
9     return NextResponse.redirect(new URL("/auth/login", request.url));
10  }
11
12  // return NextResponse.redirect(new URL("/", request.url));
13  // return NextResponse.next();
14 }
15
16 export const config = {
17   matcher: ["/produk", "/about"],
18 }

```

useEffect

```

pemograman_framework - index.tsx

8  const [isLogin, setsLogin] = useState(false);
9    useEffect(() => {
10      // Redirect ke halaman login jika belum login
11      if (!isLogin) {
12        window.location.href = "/auth/login";
13      }
14    }, [isLogin]);

```

Perbandingan

Fitur	useEffect (Client side)	Middleware (Server side)
Lokasi Eksekusi	Browser pengguna, setelah halaman dimuat).	Server, sebelum halaman dikirim.
Keamanan	Rendah. Konten halaman sempat terunduh dan mungkin "berkedip" sebelum redirect.	Tinggi. Request dicegat sebelum mencapai browser.
Pengalaman Pengguna	Ada jeda/loading sebelum dialihkan.	Pengalihan instan tanpa memuat halaman tujuan asli.
Akses ke API	Bisa akses window, localStorage, document.	Tidak ada akses window. Hanya bisa akses cookies & headers.
Skalabilitas	Harus dipasang di setiap file komponen.	Terpusat di satu file untuk banyak rute sekaligus (matcher).

G. PERTANYAAN ANALISIS

- Mengapa middleware lebih aman dibanding useEffect?
 Karena pengecekan terjadi di sisi server sebelum konten halaman dikirim ke browser, sehingga mencegah kebocoran data sensitif ke client.
- Mengapa middleware tidak menimbulkan glitch?
 Karena proses pengalihan (redirect) dilakukan pada level HTTP sebelum browser sempat merender komponen UI apa pun.
- Apa risiko jika semua halaman diproteksi tanpa pengecualian?

Pengguna tidak akan bisa mengakses halaman login atau aset publik, sehingga menciptakan infinite redirect loop yang merusak aplikasi.

4. Kapan middleware tidak diperlukan?

Jika logika proteksi hanya bersifat opsional untuk perubahan UI kecil atau jika data yang dicek hanya tersedia di localStorage yang tidak bisa dibaca server.

5. Apa perbedaan middleware dan API route?

Middleware berfungsi sebagai pencegat (interceptor) permintaan sebelum mencapai tujuan, sementara API route adalah tujuan akhir untuk memproses logika data dan mengirimkan respon.

LAPORAN JOBSHEET 14: SISTEM AUTENTIKASI - PROTEKSI ROUTE MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

✂ C. LANGKAH PRAKTIKUM

Bagian 1 – Install NextAuth

```
> npm install next-auth --force
npm warn using --force Recommended protections disabled.

added 15 packages, and audited 486 packages in 5s

158 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

Bagian 2 – Konfigurasi API Auth

Buat file dan folder pada folder pages/api/auth/[...nextauth].ts

Modifikasi file [...nextauth].ts:

```

pemograman_framework - [...nextauth].ts
1  import NextAuth, {NextAuthOptions} from "next-auth";
2  import CredentialsProvider from "next-auth/providers/credentials";
3
4  export const authOptions: NextAuthOptions = {
5    session: {
6      strategy: "jwt",
7    },
8    secret: process.env.NEXTAUTH_SECRET,
9    providers: [
10     CredentialsProvider({
11       name: "Credentials",
12       credentials: {
13         fullname: {label: "Full Name", type: "text"},
14         email: {label: "Email", type: "email"},
15         password: {label: "Password", type: "password"},
16       },
17
18       async authorize(credentials) {
19         const user : any = {
20           id: "1",
21           email: credentials?.email,
22           password: credentials?.password,
23           fullname: credentials?.fullname,
24         }
25         if(user){
26           return user;
27         }else{
28           return null;
29         }
30       }
31     })
32   ],
33   callbacks:{
34     async jwt({token, account, profile, user}:any){
35       if(account?.provider === "credentials" && user){
36         token.email = user.email
37       }
38       return token;
39     },
40     async session({session, token}:any){
41       if(token.email){
42         session.user.email = token.email;
43       }
44       return session;
45     },
46   },
47 };
48
49 export default NextAuth(authOptions);

```

Bagian 3 – Tambahkan Secret

Buka file .env.local dan tambahkan code pada line 12

o NEXTAUTH_SECRET=RANDOM_BASE64_STRING

o Untuk mendapatkan nilai RANDOM_BASE64_STRING gunakan generator

RANDOM_BASE64_STRING seperti <https://www.convertsimple.com/randombase64-generator/>

```

pemograman_framework - .env.local
10 // gunakan website https://generate-secret.vercel.app/64 untuk membuat secret key yang panjang dan acak
11 // lalu masukkan kedalam variabel NEXTAUTH_SECRET
12 NEXTAUTH_SECRET=94dd5597dcda66f44a6e6302556e5e802fc899cc0d8e5d8db627dc3988859

```

Bagian 4 – Tambahkan SessionProvider

Buka file _app.tsx dan modifikasi:

```

1 import '@styles/globals.css'
2 import type { AppProps } from 'next/app'
3 import Navbar from '@components/layouts/navbar'
4 import AppShell from '@components/layouts/Appshell';
5 import {SessionProvider} from "next-auth/react";
6
7 export default function App({ Component, pageProps: {session, ...pageProps} }: AppProps) {
8   return (
9     <SessionProvider session={pageProps.session}>
10     <AppShell>
11       <Component {...pageProps} />
12     </AppShell>
13   </SessionProvider>
14
15   );
16 };
17

```

Bagian 5 – Tambahkan Tombol Login & Logout

o Buka index.tsx pada folder component/navbar:

Modifikasi file index.tsx pada line 10 dan 2

[illegible]

Buka file file navbar.module.scss tambahkan code pada line 9

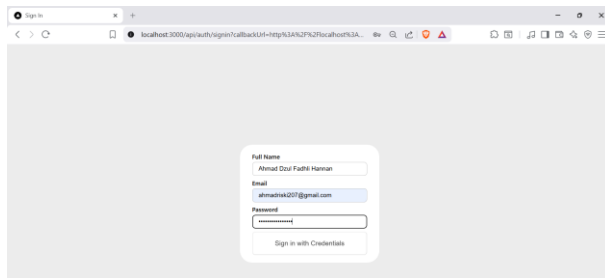
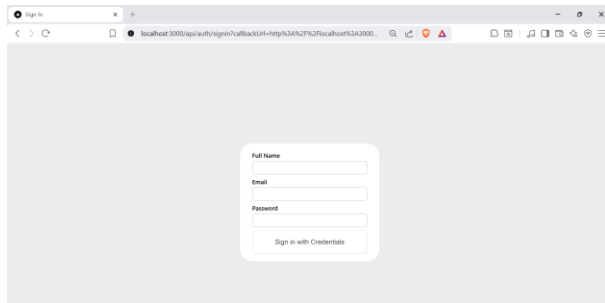
● ● ● pemograman_framework - navbar.module.css

```
1 .navbar {
2   width: 100%;
3   height: 60px;
4   background-color: #333;
5   color: white;
6   display: flex;
7   align-items: center;
8   padding: 0 20px;
9   font-size: 25px;
10  font-weight: 600;
11  justify-content: space-between;
12 }
```

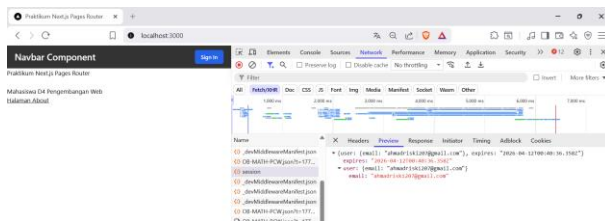
Jalankan <http://localhost:3000/>



Jika di klik sign in maka akan muncul dan isikan textbox masing. Setelah itu klik button sign in dan setelah diklik maka akan kembali ke halaman localhost



Setelah berhasil login maka akan muncul session



Untuk dapat menangkap data pada session maka tambahkan code sebagai berikut :



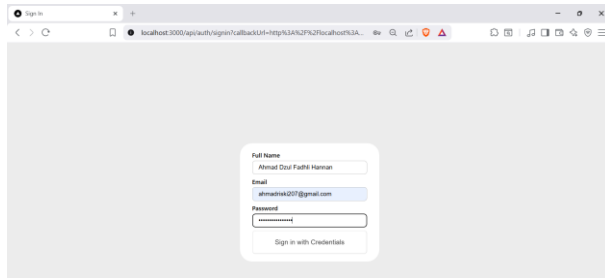
Uji coba sign in dan sign out

o Jalankan Kembali npm run dev

o Jalankan localhost



Klik sign in dan isikan textboxnya



Maka akan muncul tombol signout

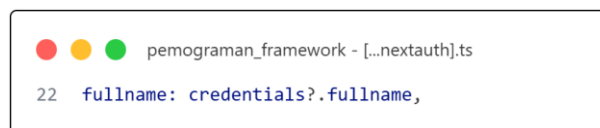


Ketika user klik signout maka akan kembali sign in

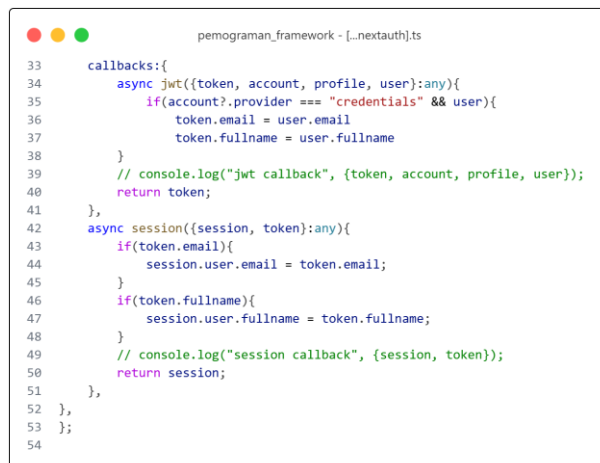


D. MENAMBAHKAN DATA TAMBAHAN (FULL NAME)

o Buka file [...nextauth].js dan tambahkan code pada line 22



Pada callbacks modifikasi codenya menjadi berikut :



Modifikasi navbar.module.scss

```
pemograman_framework - navbar.module.css

1  .navbar {
2    width: 100%;
3    height: 70px;
4    padding: 0 60px;
5
6    display: flex;
7    align-items: center;
8    justify-content: space-between;
9
10   background: linear-gradient(135deg, #0f172a, #1e293b);
11   color: #ffffff;
12
13   box-shadow: 0 4px 20px rgba(0, 0, 0, 0.2);
14   border-bottom-left-radius: 16px;
15   border-bottom-right-radius: 16px;
16 }
17
18 /* Logo / Brand */
19 .navbar__brand {
20   font-size: 22px;
21   font-weight: 700;
22   letter-spacing: 0.5px;
23 }
24
25 /* Right Section */
26 .navbar__right {
27   display: flex;
28   align-items: center;
29   gap: 32px;
30 }
31
32 /* Welcome text */
33 .navbar__user {
34   font-size: 16px;
35   font-weight: 500;
36   opacity: 0.9;
37 }
38
39 /* Button */
40 .navbar__button {
41   padding: 8px 18px;
42   border-radius: 999px;
43   border: none;
44   cursor: pointer;
45   font-weight: 600;
46   transition: all 0.3s ease;
47 }
48
49 /* Sign In */
50 .navbar__button--primary {
51   background-color: #3b82f6;
52   color: white;
53 }
54
55 .navbar__button--primary:hover {
56   background-color: #2563eb;
57   transform: translateY(-2px);
58 }
59
60 /* Sign Out */
61 .navbar__button--danger {
62   background-color: #ef4444;
63   color: white;
64 }
65
66 .navbar__button--danger:hover {
67   background-color: #dc2626;
68   transform: translateY(-2px);
69 }
70
71 /* Responsive */
72 @media (max-width: 768px) {
73   .navbar {
74     padding: 0 20px;
75   }
76
77   .navbar__brand {
78     font-size: 18px;
79   }
80
81   .navbar__user {
82     display: none;
83   }
84 }
```

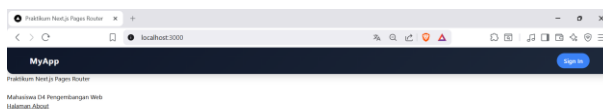
Modifikasi index.tsx pada folder components/layouts/navbar

```

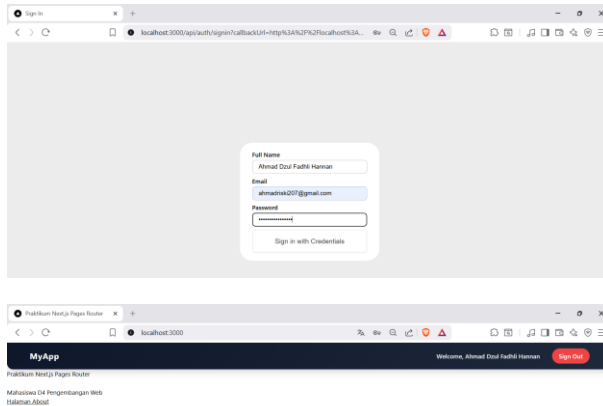
1 import styles from './navbar.module.css';
2 import { signIn, signOut, useSession } from 'next-auth/react';
3
4 const Navbar = () => {
5   const { data } = useSession();
6   // const (data: session) = useSession();
7   // console.log("session:", session);
8   return (
9     <div className={styles.navbar}>
10       <div className={styles.navbar_brand}>
11         MyApp
12       </div>
13       <div className={styles.navbar__right}>
14         {data ? (
15           <div className={styles.navbar__user}>
16             Welcome, {data.user?.fullname}
17           </div>
18           <button
19             className={styles.navbar__button} ${styles["navbar__button--danger"]}
20             onClick={() => signOut()}
21           >
22             Sign Out
23           </button>
24         ) : (
25           <button
26             className={styles.navbar__button} ${styles["navbar__button--primary"]}
27             onClick={() => signIn()}
28           >
29             Sign In
30           </button>
31         )}
32       </div>
33     </div>
34   );
35 };
36
37 export default Navbar;

```

Jalankan browser pada localhost



Lakukan sign in



E. PROTEKSI HALAMAN PROFILE

Buat Halaman Profile

o pages/profile/index.tsx dan modifikasi file index.tsx

```
1 import {useSession} from "next-auth/react";
2
3 const HalamanProfile = () => {
4   const {data}:any = useSession();
5   return (
6     <div>
7       <h1 className="text-2xl font-bold pl-4">Halaman Profile</h1><br />
8       <h1 className="text-2xl font-bold pl-4">Selamat Datang {data?.user?.fullname}</h1><br />
9     </div>
10  )
11 }
12
13 export default HalamanProfile;
```

jalankan browser



Buat Middleware Authorization

o Buat file withAuth.ts dan folder dengan nama middleware di src

Modifikasi withAuth.ts

```
1 import { getToken } from "next-auth/jwt";
2 import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";
3
4 export default function withAuth(
5   middleware: NextMiddleware,
6   requireAuth: string[] = [],
7 ) {
8   return async (req: NextRequest, next: NextFetchEvent) => {
9     const pathname = req.nextUrl.pathname;
10
11     if (requireAuth.includes(pathname)) {
12       const token = await getToken({
13         req,
14         secret: process.env.NEXTAUTH_SECRET,
15       });
16
17       if (!token) {
18         const loginUrl = new URL("/login", req.url);
19         return NextResponse.redirect(loginUrl);
20       }
21     }
22     return middleware(req, next);
23   }
24 }
```

Modifikasi file middleware.ts

```
1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3 import { getToken } from "next-auth/jwt";
4
5 export async function middleware(request: NextRequest) {
6   const token = await getToken({
7     req: request,
8     secret: process.env.NEXTAUTH_SECRET,
9   });
10
11   if (token) {
12     return NextResponse.next();
13   }
14
15   return NextResponse.redirect(new URL("/", request.url));
16 }
17
18 export const config = {
19   matcher: ["/produk", "/about", "/lapar", "/profil"],
20 }
```


Jika user mengarahkan ke halaman profile tidak akan bisa, user akan diarahkan ke alamat localhost

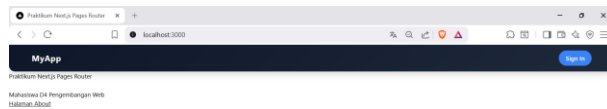
F. PENGUJIAN

Uji 1 – Belum Login

Akses:

/profile

Hasil: Redirect ke home

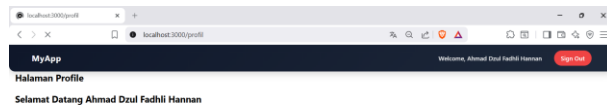


Benar

Uji 2 – Sudah Login

Login terlebih dahulu → Akses /profile

Hasil: Bisa masuk



Benar

Uji 3 – Logout

Klik Sign Out → Akses /profile

Hasil: Tidak bisa masuk



Benar, sama dengan saat belum login

G. ALUR LOGIN NEXTAUTH

User klik Sign In

↓

NextAuth tampilkan form credentials

↓

Authorize() dijalankan

↓

JWT dibuat

↓

Session disimpan

↓

Frontend akses useSession()

H. TUGAS PRAKTIKUM

1. Implementasikan login menggunakan Credentials Provider.

```
pemograman_framework - _app.tsx
1 import '@/styles/globals.css'
2 import type { AppProps } from 'next/app'
3 import Navbar from '@components/layouts/navbar'
4 import AppShell from '@components/layouts/Appshell';
5 import {SessionProvider} from "next-auth/react";
6
7 export default function App({ Component, pageProps: {session, ...pageProps} }: AppProps) {
8   return (
9     <SessionProvider session={pageProps.session}>
10     <AppShell>
11       <Component {...pageProps} />
12     </AppShell>
13   </SessionProvider>
14 );
15 };
16
17
```

2. Tambahkan field full name.

o Buka file [...nextauth].js dan tambahkan code pada line 22

```
pemograman_framework - [...nextauth].ts
22 fullname: credentials?.fullname,
```

Pada callbacks modifikasi codenya menjadi berikut :

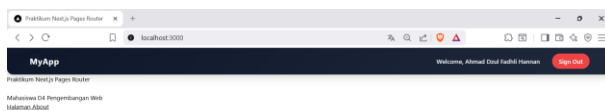
```
pemograman_framework - [...nextauth].ts

33   callbacks:{
34     async jwt({token, account, profile, user}:any){
35       if(account?.provider === "credentials" && user){
36         token.email = user.email
37         token.fullname = user.fullname
38       }
39       // console.log("jwt callback", {token, account, profile, user});
40       return token;
41     },
42     async session({session, token}:any){
43       if(token.email){
44         session.user.email = token.email;
45       }
46       if(token.fullname){
47         session.user.fullname = token.fullname;
48       }
49       // console.log("session callback", {session, token});
50       return session;
51     },
52   },
53 };
54
```

3. Tampilkan full name setelah login.

```
pemograman_framework - index.tsx

1  import styles from "./navbar.module.css";
2  import { signIn, signOut, useSession } from "next-auth/react";
3
4  const Navbar = () => {
5    const { data } :any = useSession();
6    // const {data: session} = useSession();
7    // console.log("session:", session);
8    return (
9      <div className={styles.navbar}>
10        <div className={styles.navbar__brand}>
11          MyApp
12        </div>
13        <div className={styles.navbar__right}>
14          <div> {
15            <div className={styles.navbar__user}>
16              Welcome, {data.user?.fullname}
17            </div>
18            <button
19              className={(`${styles.navbar__button} ${styles["navbar__button--danger"]`)}
20              onClick={() => signOut()}>
21              Sign Out
22            </button>
23          </div>
24          <div> {
25            <button
26              className={(`${styles.navbar__button} ${styles["navbar__button--primary"]`)}
27              onClick={() => signIn()}>
28              Sign In
29            </button>
30          </div>
31        </div>
32      </div>
33    );
34  };
35
36  export default Navbar;
```

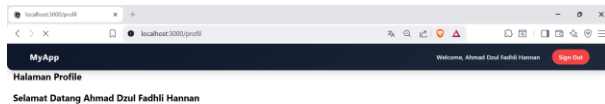


Ditampilkan di navbar

4. Buat halaman profile.

```
pemograman_framework - index.tsx

1  import {useSession} from "next-auth/react";
2
3  const HalamanProfile = () => {
4    const {data}:any = useSession();
5    return (
6      <div>
7        <h1 className="text-2xl font-bold pl-4">Halaman Profile</h1><br />
8        <h1 className="text-2xl font-bold pl-4">Selamat Datang {data?.user?.fullname}</h1><br />
9      </div>
10    )
11  }
12
13  export default HalamanProfile;
```



5. Lindungi halaman profile dengan middleware.

o Buat file withAuth.ts dan folder dengan nama middleware di src

Modifikasi withAuth.ts

```
pemograman_framework - withAuth.ts
1 import { getToken } from "next-auth/jwt";
2 import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";
3
4 export default function withAuth(
5   middleware: NextMiddleware,
6   requireAuth: string[] = [],
7 ) {
8   return async (req: NextRequest, next: NextFetchEvent) => {
9     const pathname = req.nextUrl.pathname;
10
11     if (requireAuth.includes(pathname)) {
12       const token = await getToken({
13         req,
14         secret: process.env.NEXTAUTH_SECRET,
15       });
16
17       if (!token) {
18         const loginUrl = new URL("/login", req.url);
19         return NextResponse.redirect(loginUrl);
20       }
21     }
22     return middleware(req, next);
23   }
24 }
```

Modifikasi file middleware.ts

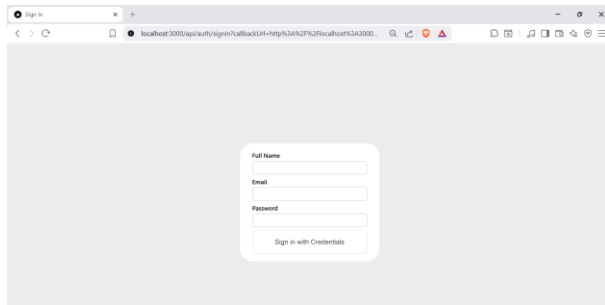
```
pemograman_framework - middleware.ts
1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3 import { getToken } from "next-auth/jwt";
4
5 export async function middleware(request: NextRequest) {
6   const token = await getToken({
7     req: request,
8     secret: process.env.NEXTAUTH_SECRET,
9   });
10
11   if (token) {
12     return NextResponse.next();
13   }
14
15   return NextResponse.redirect(new URL("/", request.url));
16 }
17
18 export const config = {
19   matcher: ["/produk", "/about", "/lapar", "/profil"],
20 }
```

Jika user mengarahkan ke halaman profile tidak akan bisa, user akan diarahkan

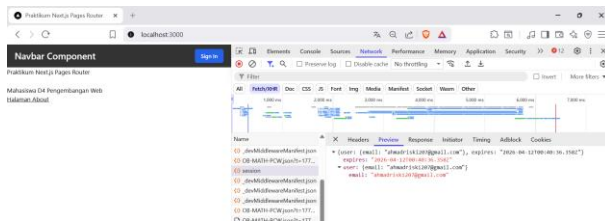
ke alamat localhost

6. Dokumentasikan:

o Screenshot login



o Screenshot session



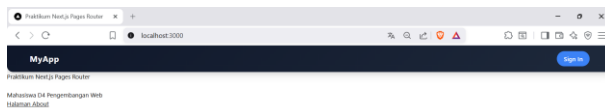
o Screenshot redirect middleware

Uji 1 – Belum Login

Akses:

/profile

Hasil: Redirect ke home

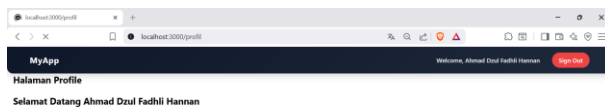


Benar

Uji 2 – Sudah Login

Login terlebih dahulu → Akses /profile

Hasil: Bisa masuk



Benar

Uji 3 – Logout

Klik Sign Out → Akses /profile

Hasil: Tidak bisa masuk



Benar, sama dengan saat belum login

I. PERTANYAAN ANALISIS

1. Mengapa session menggunakan JWT?

Karena bersifat *stateless*, sehingga server tidak perlu menyimpan data sesi di database dan cukup memverifikasi token yang dibawa oleh client untuk mengenali user.

2. Apa perbedaan authorize() dan callback jwt()?

authorize() berfungsi memvalidasi kredensial saat login, sementara callback jwt() bertugas memanipulasi atau menambahkan data ke dalam token setelah login berhasil.

3. Mengapa middleware perlu getToken()?

Untuk mengambil dan mendekripsi data user dari cookie secara cepat guna menentukan apakah sebuah request memiliki izin untuk mengakses halaman yang diproteksi.

4. Apa risiko jika NEXTAUTH_SECRET tidak digunakan?

Token JWT menjadi sangat mudah dipalsukan oleh penyerang karena tidak ada kunci rahasia untuk menandatangani dan memverifikasi integritas data.

5. Apa perbedaan autentikasi dan otorisasi dalam sistem ini?

Autentikasi memastikan "siapa Anda" (identitas user), sedangkan otorisasi menentukan "apa yang boleh Anda lakukan" (hak akses ke halaman atau fitur tertentu).